Outlook Plugin Architecture Discussion

USER

i want to enhance outlook by a custom button/option to store the attachment at some specific location. What is the right approach? A plugin/extension? It should work on outlook 365, outlook 2024 and outlook 2019

ASSISTANT

To determine the right approach, we have to look at Microsoft's current transition phase with Outlook. The correct path splits based on two critical factors: **where** the file needs to go (local disk vs. server/cloud) and **which version** of Outlook 365 you are targeting (Classic desktop vs. the "New Outlook").

Here is the breakdown of the two architectural paths.

1. The COM/VSTO Add-in (Legacy Windows Approach)

If your goal is to save the attachment to a hardcoded local drive or network share (e.g., `C:\Attachments` or `Z:\Invoices`), and you want it to happen silently without browser download prompts, a COM/VSTO add-in is historically the way to do this.

- **How it works:** You use the `Microsoft.Office.Interop.Outlook` library in C#/.NET and call the `Attachment.SaveAsFile(path)` method.
- **Compatibility:** Works perfectly on the classic desktop versions of Outlook 2019, Outlook 2024, and Outlook 365 (Classic).
- **The Catch:** This is a dead end for modern development. Microsoft's "New Outlook for Windows" is essentially a web wrapper and **completely drops support** for COM/VSTO add-ins. Furthermore, if you or your team operate on Apple Silicon or any Mac hardware, COM add-ins will not load at all.

2. The Office Web Add-in (Modern Cross-Platform Approach)

If this button is part of a larger enterprise helper or an automated workflow—such as feeding attachments into a pipeline for analysis—you should build an Office Web Add-in.

- **How it works:** The add-in runs in a sandboxed browser environment inside Outlook using JavaScript, TypeScript, HTML, and `Office.js`. It extracts the attachment and sends it to a backend server or cloud storage via an API.

- **Compatibility:** This is the future-proof route. It works across Outlook 2019, 2024, 365 (Classic & New), Mac, and Web.
- **The Catch:** Because it runs in a browser sandbox, **it cannot natively write to a specific local file path**. If you try to trigger a local save directly, it will use the browser's standard download behavior, which usually prompts the user or saves to the default Downloads folder.

How to handle local saves with a Web Add-in

If you absolutely must use the modern Web Add-in framework but still need files to land in a specific local folder silently, you have to build a bridge:

1. **The Local Daemon:** Run a lightweight background service (e.g., a Python or Node script) on the target machine listening on a `localhost` port.
2. **The Add-in Payload:** The Web Add-in grabs the attachment data and sends an HTTP POST request to that local service.
3. **The Write:** The local daemon receives the payload and writes the file directly to the specific directory.

Summary Comparison

Feature	COM / VSTO Add-in	Office Web Add-in
Tech Stack	C#, .NET	JavaScript, TypeScript, HTML
Outlook 2019/2024 Support	Yes (Windows only)	Yes
Outlook 365 (Classic) Support	Yes (Windows only)	Yes
"New" Outlook / Web Support	**No**	Yes
Mac Support	**No**	Yes
Direct Local File System Access	Yes	**No** (Requires local daemon)

Recommendation

If you need a quick tool strictly for Windows users on classic Outlook to dump files to a local folder, VSTO is the path of least resistance. However, if you are building an architecture meant to last, routing data to a centralized location, or needing it to work across mixed operating systems, the **Office Web Add-in** is the correct choice.

Are you aiming to save these attachments strictly to local hard drives, or are you routing them to a backend server for further processing?

USER

ideally most of the work is done in some kind of backend. The plugin would only be a frontend/ user-interface without much logic.

ASSISTANT

That is exactly the right mindset for modern architecture. By keeping the logic out of the client, you avoid deployment headaches and ensure the add-in remains a lightweight trigger—making it an ideal Proof-of-Concept for a broader enterprise helper or automated pipeline.

Given this "thin client, heavy backend" strategy, the **Office Web Add-in** is absolutely the right approach.

1. The Thin Client (Outlook Web Add-in)

The plugin acts purely as the user interface. It is a small web application (HTML/JavaScript/ TypeScript) hosted on a web server.

- **The Action:** The user clicks your custom button in the Outlook ribbon.
- **Data Extraction:** The add-in uses the `Office.js` library—specifically a method like `Office.context.mailbox.item.getAttachmentContentAsync()` —to grab the base64-encoded attachment data.
- **The Handoff:** The frontend makes a standard HTTP POST request to your backend API, passing the attachment payload along with any necessary metadata (like the sender, subject, or user ID). It then shows a simple "Success" or "Error" notification to the user.

2. The Heavy Backend

This is where the actual work happens. You can build this in Python, Node.js, Go, or any language you prefer.

- **Processing:** The backend receives the HTTP request.
- **Execution:** It handles all the complex logic: validating the file type, applying business rules, and writing the file to the target location.
- **Extensibility:** Because the backend is entirely decoupled from Outlook, you can easily plug this into other tools.

3. The Graph API Route (For Large Files)

If you anticipate users sending massive attachments and you want to avoid passing heavy base64 strings through the client's browser memory, you can use a slightly different flow:
Instead of downloading the file, the Web Add-in just sends the email's **Item ID** and an **Identity**

Token to your backend. Your backend then uses the Microsoft Graph API to reach directly into the Microsoft 365 cloud, fetch the attachment, and save it.

USER

Yes, i think the frontend should only send commands to the backend, not any kind of real attachment data. The backend will have some kind of email access (e.g. via IMAP) to the mailbox. But the approach should also work with PST files. Would that mean that both, frontend and backend need to access the PST file in parallel? is that even possible?

ASSISTANT

That is a very clean, modern architectural pattern for server-based mail... However, introducing PST files throws a massive wrench into this design.

To answer your direct question: **No, parallel access to a PST file is not possible.**

When Outlook is running, it places an exclusive, unyielding lock on the PST file at the OS level. If your backend attempts to read the PST file while Outlook is open, it will immediately hit a file sharing violation.

1. The "Push" Model (Modern Web Add-in Hybrid)

If you are using Exchange/M365, but users are manually dragging emails into local PST archives attached to their profile, you must rely on the client to do the extraction. The frontend grabs the base64 attachment data and **pushes** the payload to the backend.

2. The Native Process Model (VSTO / COM Add-in)

If the environment relies heavily on PSTs—especially without an Exchange server—you are firmly in the legacy Windows domain. A VSTO (C#/.NET) add-in runs *inside* the Outlook application process and bypasses the file-lock issue entirely.

USER

Yes, i think we can drop this requirement. Is it possible to write a single Add-in that is supported for outlook 2019, 2024 and 365?

ASSISTANT

Yes, absolutely. In fact, that is the primary advantage of the Office Web Add-in model. You write a single codebase, host it once on your web server, and distribute a single manifest file to all users.

The mechanism that makes this cross-version compatibility possible without maintaining multiple codebases is Microsoft's system of **Requirement Sets**. Outlook 2019 generally only supports Mailbox Requirement Sets up to **1.7 or 1.8**. Because Outlook 2019 is your baseline, the core functionality must be designed to operate within these limits.

USER

What is the programming language for the addin?

ASSISTANT

An Office Web Add-in is fundamentally just a web application embedded inside a browser control within Outlook. Therefore, you build it using standard web development languages: **HTML, CSS, and JavaScript or TypeScript**.

While you can write the logic in standard JavaScript, **TypeScript** is the industry standard for modern Office Add-ins.

USER

so even outlook 2019 supports addins written in typescipt?

ASSISTANT

To be technically precise: **No version of Outlook (or any web browser) runs TypeScript directly.** TypeScript is a developer tool, not a runtime language.

When you build an Office Web Add-in using TypeScript, it goes through a build process (transpilation) before it ever touches Outlook. Outlook 2019 runs the compiled JavaScript.

The Outlook 2019 "Gotcha": Because Outlook 2019 relies on older web engines (often IE11 or legacy EdgeHTML), you must configure your compiler to **target ES5** (older JavaScript) and use "polyfills".

USER

In case i will create a completely different frontend, e.g. web-based for gmail or IMAP, can i re-use the backend without any adjustments?

ASSISTANT

Yes, you absolutely can, provided you enforce a strict boundary between your API layer and your core business logic.

Scenario A: 100% Reusable (The "Push" Model)
If your frontend extracts the attachment and sends it as a base64 payload to the backend, your backend is completely client-agnostic. Zero backend adjustments required.

Scenario B: 90% Reusable (The "Fetch" Model)
If your frontend only sends an ID and an Auth Token (relying on the backend to fetch the file), you will need to build an **abstraction layer** (often referred to as an adapter pattern) for different clients (e.g., M365 vs Gmail).

USER

My idea is a design of at least these modules:
- the frontend (e.g. outlook addin or new webapp) which decides which mail's attachments shall be archived. Send an archive request to an archive-manager
- a storage-backend that stores the attachment at some target location (may be even a database)
- a mail-client backend that has access to the mailbox (e.g. IMAP, GMAIL, ...)
- an archive-manager that receives commands from the frontend which mail needs to be handled. Optionally receives the information whether all attachments or only a subset shall be archived
- a dispatcher that decides where to archive the attachment (based on mail sender, mail content etc.)
Does this make sense? Maybe more modules needed? Or less?

ASSISTANT

This architecture makes perfect sense. What you have described is a textbook example of a **Ports and Adapters (Hexagonal) Architecture**. It enforces strict boundaries between your core logic and your external dependencies.

What Should Be Added?

- **An Analyzer / Extraction Module (The Agentic Layer):** To extract text via OCR or parsing and pass it through an LLM inference to classify the document and extract key-value pairs for the Dispatcher.
- **A State / Queue Manager:** To track the state of a job (Pending, Fetching, Stored) so you can handle retries gracefully without dropping data.

Can We Consolidate?

If you are building a Proof-of-Concept, you can consolidate them into **three physical services** while maintaining the five **logical modules** in your code: Frontend Service, Core API Service, and Adapter Service.